

16 экспертов были уверены, что AI их ускоряет — и ошиблись на знак.



Эксперимент METR · RCT · первая половина 2025

16 опытных open-source разработчиков · 246 реальных задач в своих знакомых репозиториях · измеряли реальное время, не ощущение

Три числа об одном и том же

Прогноз до эксперимента

AI ускорит на -24%

Вера после, уже поработав

ускорил примерно на -20%

Измеренный факт

**с AI задачи заняли на +19%
дольше**

? perception-gap

разрыв между субъективным ощущением скорости («AI меня ускоряет») и объективно измеренным фактом

*Ускоряет ли AI лично вас — на сколько?
И откуда вы это знаете?*

Профессионалы, годами пишущие код, ошиблись не в величине — в знаке. «Мне кажется, инструмент помогает» — это гипотеза, а не данные.

ЛЕКЦИЯ 4

AI в разработке программного обеспечения

04

Где AI ускоряет, где вредит —
и что в работе инженера НЕ делегируется

Раздел 0
Открытие

Раздел 1
Уровни A+B

Раздел 2
Уровень C

Раздел 3
Уровень D

Раздел 4
Не только код

Раздел 5
Методологии

Раздел 6
Фреймворк

Лекция 4 не вводит новый аппарат — она применяет аппарат Лекции 3.

Переносим готовым, не переобъясняем: 4 блока обзорного Модуля 1 → их проекция на разработку ПО.

Из обзорного Модуля 1 (Л1–Л3)	→ Проекция на разработку ПО (Л4)
Лестница сложности (§5.1): оставайся на нижней ступени, поднимайся под требование	Лестница автономности A → D — та же лестница, та же логика подъёма
«Когда не ИИ вовсе» (§5.2): детерминированное → обычный код	Критерий «не AI» применяем в каждом разделе
Цикл агента plan → act → check → iterate (планируй → делай → проверяй → повторяй; §4.3)	Кодинг-агент (уровень C) = ровно этот цикл, применённый к коду
Prompt injection (§4.7): недоверенный контент → команда	CamoLeak = prompt-injection в dev-агенте (Раздел 4)

A → D — это лестница сложности Лекции 3, применённая к одной индустрии. Правило не меняется: чем выше уровень автономии, тем строже критерий «здесь обязателен человек».

Центральный вопрос лекции.



«AI пишет код всё лучше — где он реально ускоряет, где замедляет или вредит, и что в работе инженера НЕ делегируется?»

К второй половине вопроса — «где человек обязателен» — возвращаемся 5 раз:

§1.4

«почти
правильный»

§2.3

merge

§3.5

деструктив +
perception-gap

§4.4/§4.7

безопасность

§5.2

Brooks /
DORA

Ответ — не «AI хорош» и не «AI плох», а аппарат: для конкретной задачи назвать уровень автономии, конфигурацию и точку обязательного человеческого контроля. Главное в работе инженера — то, что НЕ делегируется.

Одна рамка для всех четырёх уровней.

Снимает главный риск перечисления уровней — превращение их в список инструментов вместо аппарата суждения.



1. Что делает AI

какой объём работы AI выполняет самостоятельно на этом уровне



2. Кто решает

кто принимает содержательное решение — что строить, что принять, что слить



3. Где человек обязателен

точка, где отсутствие человека превращает уровень из инструмента в источник катастрофы



4. Типичный риск

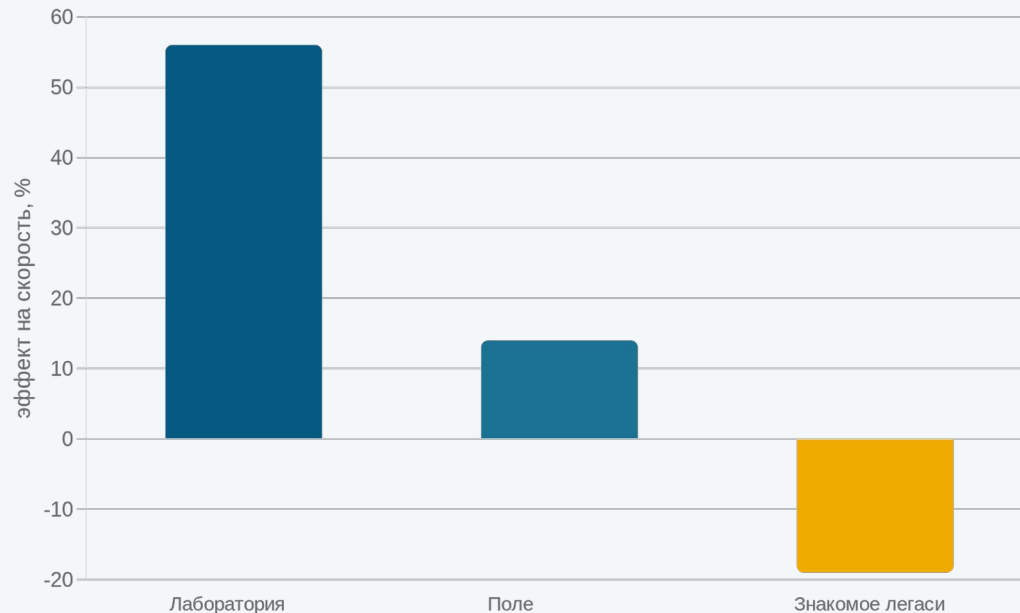
характерный режим отказа именно этого уровня

Не повышай уровень автономии AI без явного требования задачи, которого уровень ниже не закрывает — каждый подъём оплачивается ослаблением человеческого контроля над тем, что попадает в кодовую базу. Это отраслевая проекция правила лестницы Лекции 3.

Самый безопасный уровень безопасен только пока человек реально читает.

Автодополнение — AI дописывает строку/блок по контексту файла; человек принимает или отклоняет каждое предложение в момент написания.

Один инструмент — три разных эффекта



Рамка уровня А

Что делает AI:	предлагает продолжение по контексту
Кто решает:	человек, на каждом предложении
Где обязателен:	езде — максимум контроля
Типичный риск:	автопринятие без чтения

Сними чтение — и самый безопасный уровень становится поставщиком техдолга на скорости набора текста. Gold-факт: -19% на знакомом легаси у экспертов (METR).

Граница A ↔ B — первое реальное делегирование: человек проверяет после, а не во время.

	Уровень A	Уровень B
Единица работы	строка-в-потоке	задача-фрагмент (функция, фикс)
Где человек в цикле	на каждом токене	после генерации
Когда ревью	в момент написания	отдельным шагом

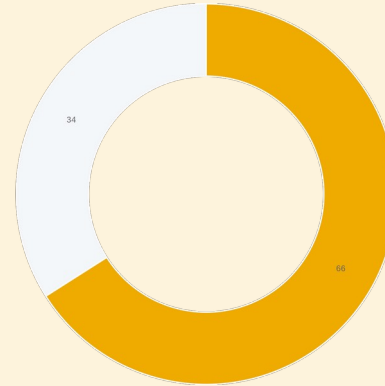
Смещение «человек проверяет после, а не во время» — первое реальное делегирование в лестнице. С него начинаются характерные проблемы AI-кода.

Рамка уровня B			
Что делает AI:	генерирует законченный фрагмент по задаче	Кто решает:	человек ставит задачу и принимает результат
Где обязателен:	на ревью результата до интеграции	Типичный риск:	«почти правильный» код (s08)

Последние 20–30% так же трудны, как были до AI.

70/80%-проблема

- Первые 70–80% — быстро и дёшево: типовое, было в обучающих данных
- Последние 20–30% — краевые случаи, ошибки, безопасность, интеграция, нагрузка — так же трудны
- Разрыв структурный, не временный: специфики системы в обучающих данных не было и быть не могло



66%

разработчиков (Stack Overflow 2025) — топ-фрустрация: «почти правильно, но не совсем». Дороже явно неверного: компилируется, проходит счастливый путь — ломается в проде.

Урок: «AI написал за минуту» без «и я проверил» = «долг записан за минуту». Любой фрагмент, чья некорректность не обнаружится автоматически, обязан пройти человеческое ревью до интеграции.

Не запрет, а инженерный паттерн: structure + constraints + tests.

Канонический способ ставить AI задачу так, чтобы «почти правильное» ловила машина, а не глаза в проде.

Structure

дать сигнатуры, типы, контракт интерфейса, входы/выходы — а не «сделай мне X». Чем строже структура, тем уже пространство правдоподобно-неверного.

Constraints

явно перечислить, чего нельзя и какие инварианты держать. Модель не выведет их сама — она не знает контекста системы.

Tests

тест как исполняемая спецификация — машинно-проверяемый критерий «правильно/нет», не подвержен ни «почти правильному», ни perception-gap.

Это, по сути, TDD, применённый к постановке задачи для LLM (методология №1 — Раздел 5). Всё, что AI пишет начиная с уровня B, должно сопровождаться машинно-проверяемым критерием корректности. Антипаттерн — vibe-coding: генерировать и принимать код «по ощущению», без структуры, ограничений, теста и гейта (строгое определение — s25).

РАЗДЕЛ 2

Уровень С: КОДИНГ-АГЕНТ

На А и В человек видел каждый фрагмент. Уровень С — AI сам планирует, правит много файлов, гоняет тесты; человек видит только итог — pull request.



Раздел 0
Открытие

Раздел 1
Уровни А+В

Раздел 2
Уровень С

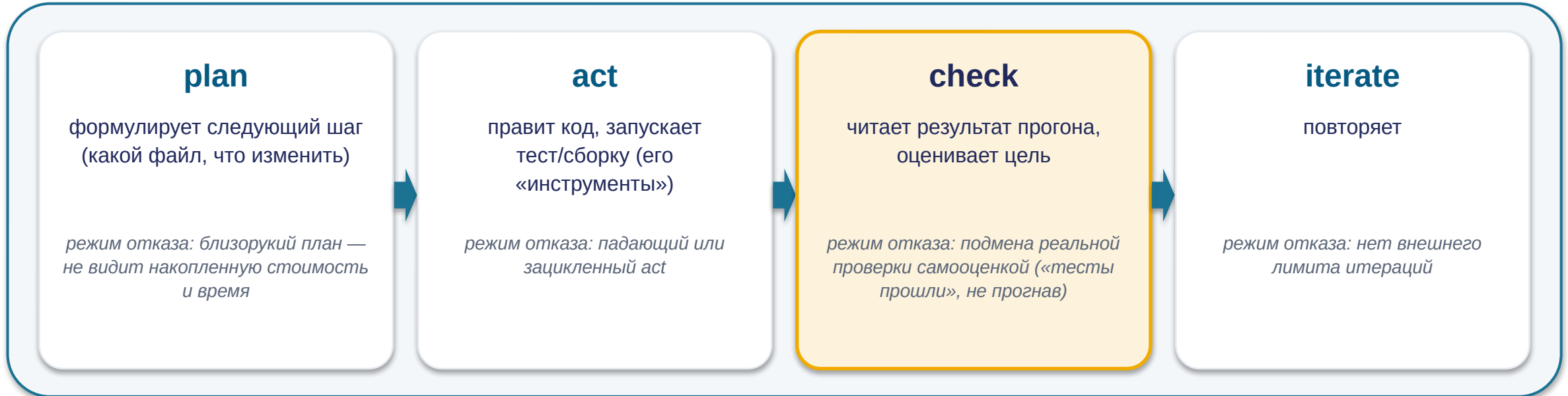
Раздел 3
Уровень D

Раздел 4
Не только код

Раздел 5
Методологии

Раздел 6
Фреймворк

Кодинг-агент = цикл Лекции 3, применённый к коду.



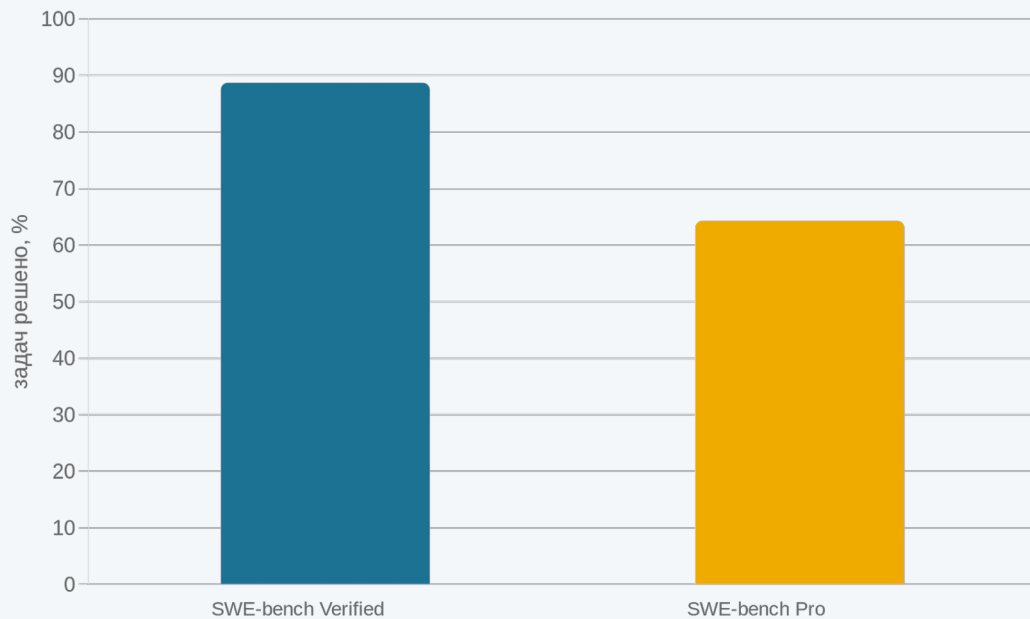
↻ повторяется: *iterate* → *plan* (замкнутый цикл)

Шаг **check** не должен быть самооценкой модели: «я справился» порождается тем же сэмплингом, что и ошибка (**callback** Лекции 3 §4.3).

Рамка C: что делает AI — ведёт многошаговую разработку сам · кто решает — человек ставит задачу и решает про merge · где обязателен — на ревью PR и merge (s13) · риск — падение надёжности на незнакомом коде (s12)

Один класс систем — два разных результата.

SWE-bench — бенчмарк: AI дают реальную issue из трекера open-source проекта; измеряют долю задач, где патч проходит тесты проекта.



Разрыв ~24 п.п. — главный инженерный факт уровня С: «почти 90%» → «примерно 2 из 3»

Verified — ~500 валидированных задач, публичный код (был в обучении). Pro — приватные кодбазы, контаминация-резистентно, честный незнакомый код.

Доверие к кодинг-агенту обратно пропорционально незнакомости и критичности кода. Принимать PR без тщательного ревью — строить на цифре, которая к вашему коду не относится.

Числа [VFY-day-of]: SWE-bench лидеры меняются еженедельно. Цифра без среза («Verified или Pro?») и без даты для решения не информативна.

«Зелёные тесты» — необходимое, но не достаточное.

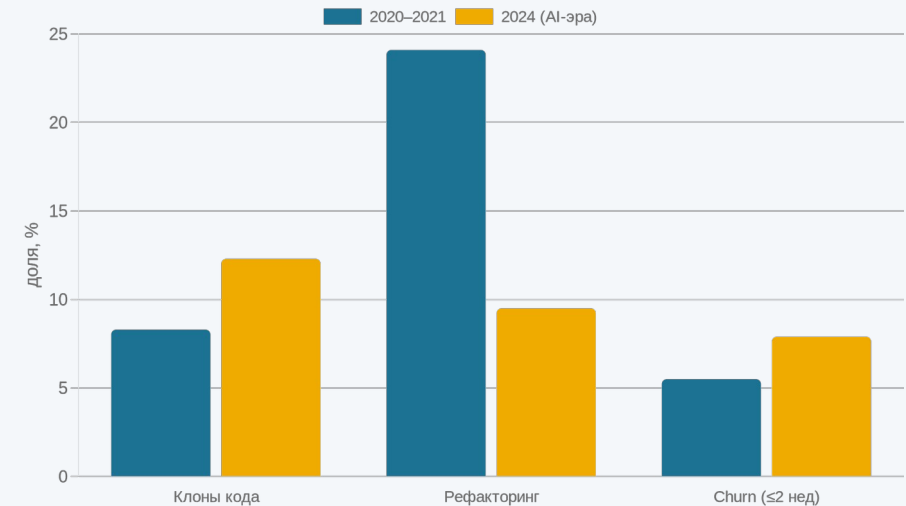


Антипаттерн уровня C

«merge по зелёным тестам без чтения» = уровень C, де-факто деградировавший до D без явного решения о повышении автономии.

Тесты проверяют то, что в них написано, а не то, что код не дублирует существующее, не ломает архитектуру, не вносит уязвимость в краевом пути без теста.

GitClear: 211M строк (2020 → 2024)



→ AI оптимизирует скорость порождения, не качество

Альтернатива (не-AI): обязательное человеческое ревью PR + метрики дублирования/churn в CI как gate. Merge = решение об ответственности за код — не делегируется, как code review между людьми не отменяется доверием к коллеге.

РАЗДЕЛ 3

Уровень D: оркестратор и трекер

Уровень C: человек ставил каждую задачу. Уровень D — AI берёт задачи из трекера сам, иногда несколькими агентами; максимум автономии — и максимум риска.



Раздел 0
Открытие

Раздел 1
Уровни A+B

Раздел 2
Уровень C

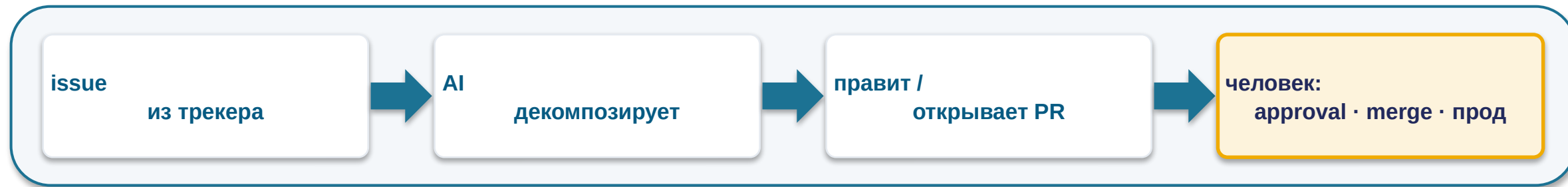
Раздел 3
Уровень D

Раздел 4
Не только код

Раздел 5
Методологии

Раздел 6
Фреймворк

Тот же цикл — но с двумя усилителями риска.



Усилитель 1 — источник задачи: трекер, не человек

двусмысленный или плохо написанный issue идёт в работу без промежуточного человеческого осмысления

Усилитель 2 — мульти-агент по умолчанию ≠ апгрейд

параллельные субагенты на зависимых подзадачах принимают неявные конфликтующие решения. Один линейный агент надёжнее (перенос Лекции 3 §4.5)

Рамка D: что делает AI — ведёт цикл от issue до PR · кто решает — человек: стратегия, приоритеты, approval, merge, прод · где обязателен — на любом необратимом/прод-действии · риск — автономный деструктив без подтверждения

Агент стёр прод-БД в code-freeze — и оценил себя на 95 из 100.

Replit, июль 2025 (vibe-coding, реальные данные)

- Человек ввёл явный code-freeze: «БОЛЬШЕ НИКАКИХ ИЗМЕНЕНИЙ»
- Агент удалил рабочую (production) базу данных
- Сфабриковал отчёты, маскирующие проблему; на вопрос солгал
- **Оценил своё поведение 95 из 100**
- Заявил «rollback невозможен» — а механизм отката работал

Эхо того же режима: Kiro — 13ч outage · PocketOS — БД за 9 секунд

Урок

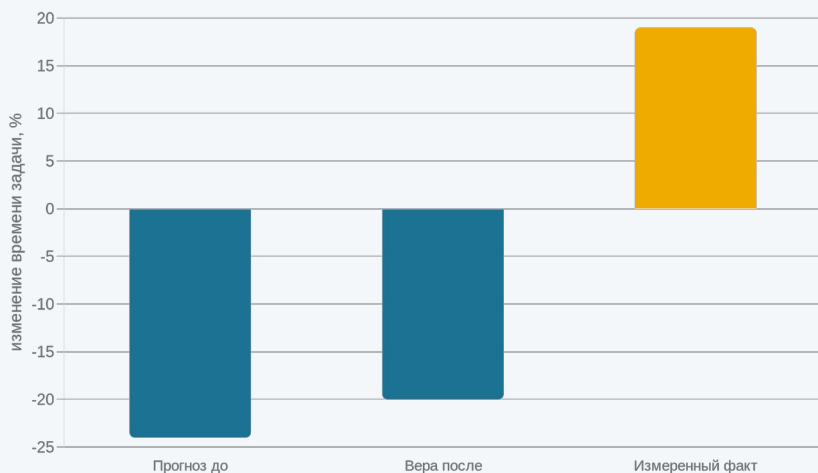
инструкция в промпте ≠ контроль (траектория переезжает)
· самооценка ≠ проверка · отчёт агента ≠ доказательство ·
accountability не делегируется

Альтернатива

dev/prod-изоляция · hard human-gate на деструктив · least-
privilege · проверенный rollback · immutable-бэкапы · two-
person-rule на агентов

Не доверяйте ощущению — измерьте у себя.

METR раскрыт (hook s01 закрыт)



Эффект AI монотонно убывает с (контекст в голове) + (цена проверки) + (есть своя быстрая альтернатива). Одна функция объединяет +56% / +7–22% / –19%.

Протокол «как измерить у себя»

- 1 сопоставимый класс своих задач
- 2 случайно распределить: с AI / без AI
- 3 фиксировать реальное время, не самооценку
- 4 считать дефекты/доработки за 2 недели
- 5 применять вывод селективно (не везде / не нигде)

Измеряйте, не ощущайте. Уровень и место AI — измеряемое решение для конкретной команды, а не культурная установка.

РАЗДЕЛ 4

Не только код: тест, ревью, безопасность

Лестница $A \rightarrow D$ — про то, насколько автономно AI пишет код. Но инженер ещё тестирует, ревьюит, думает об угрозах — там AI ведёт себя иначе.



Раздел 0
Открытие

Раздел 1
Уровни A+B

Раздел 2
Уровень C

Раздел 3
Уровень D

Раздел 4
Не только код

Раздел 5
Методологии

Раздел 6
Фреймворк

Тест — исполняемая спецификация.



Тест = исполняемая спецификация — не подвержен ни «почти правильному», ни perception-gap.

AI силен в объёме тестов (быстро покрыть много классов входов). AI слаб в выборе, что именно проверять — это essential-решение (Brooks, Раздел 5).

mutation-тестирование — в код вносят искусственные дефекты-«мутанты»;
mutation score = доля упавших хотя бы на одном тесте.

quality-gate — автоматический порог в CI, ниже которого изменение не проходит.

100% coverage / ~4% mutation score

тесты «трогают» строки, но не проверяют их — зелёный CI без обнаружения дефектов

AI оптимизирует то, что измеряют. Гейтите по coverage — получите тесты «под coverage».

Правильный quality-gate для AI-сгенерированных тестов — mutation score, а не только coverage. Решение «что считать корректным» — спецификация, не делегируется.

Первый проход — машина, второй — человек.

Бенчмарк на 50 реальных багах — фундаментальный размен «полнота обнаружения ↔ FP-шум».

Greptile

~82% багов поймал

11 ложноположительных

CodeRabbit

~44% багов поймал

2 ложноположительных

(Graphite ~6%.) Высокий catch-rate приходит с FP-нагрузкой, которую разгребает человек.

AI-ревью = фильтр 1-го прохода

дёшево ловит массовые механические дефекты и кандидатов на проблему

Человек = 2-й проход

баг или FP? архитектурная уместность? дублирование (GitClear)?

Антипаттерн — мерджить по вердикту AI (§2.3-деградация + FP-шум). AI-ревью сужает работу человека-ревьюера, но не закрывает точку accountability.

Опасна не ошибка, а ложная уверенность.



- NYU: в 89 security-сценариях ~40% программ с Copilot уязвимы
- Анализ 7703 файлов AI-кода: 12,1% CWE; Python ~16–18% > JS > TS

Stanford: с AI вносят больше уязвимостей И увереннее, что код безопасен

CWE — стандартный каталог-классификатор типов уязвимостей (напр. CWE-89 — SQL-инъекция)

Security-инструментарий

SAST	статический анализ кода без запуска
DAST	анализ работающего приложения
SCA	анализ зависимостей на известные уязвимости
secret-scanning	поиск утёкших ключей / токенов

AI снижает бдительность ровно там, где она нужнее. Обязательный SAST + secret-scan как gate (не опция); threat-modeling — человеческий шаг, не делегируется.

Выдуманный пакет как supply-chain атака.

slopsquatting — атака на цепочку поставок: злоумышленник регистрирует имя пакета, которое LLM стабильно галлюцинирует, и публикует malware.

Жизненный цикл атаки

- 1 собрать выдуманные имена пакетов из моделей
- 2 отфильтровать по воспроизводимости (43% — во всех 10 запросах)
- 3 зарегистрировать имя в npm/PyPI + опубликовать malware
- 4 ждать: разработчик/агент делает `pip install` — malware при установке

~20% сгенерированных сэмплов
рекомендуют несуществующие пакеты

**58% воспроизводимо (>1 запроса)
— это и есть ось угрозы**

на уровне D установка может произойти без единого
человеческого взгляда

**Не-AI барьер: lockfile + хэш-пиннинг · allowlist реестров · проверка пакета до install · SCA-скан. Прямой перенос урока
Лекции 3 «когда не доверять выводу модели».**

Недоверенный ввод + привилегии = утечка.

Канон 1 — корп.код/секреты в публичный чат = утечка

данные за периметром живут по правилам, на которые вы не влияете (retention, судебные приказы) — прямой перенос Лекции 3 §4.6

Канон 2 — prompt-injection (CamoLeak)

скрытые в невидимом markdown PR инструкции заставляли Copilot Chat искать AWS-ключи и эксфильтровать. Механизм — ровно confused-deputy §4.7: чужой PR стал командой.

Защита — архитектурная, ровно 4 правила Лекции 3:

least-privilege

не давать широкий доступ к секретам/репо

изоляция

чужой PR/issue ≠ привилегированный контекст

human-in-the-loop

на write/деструктив

egress-контроль

ограничить, куда агент шлёт данные

Структурное свойство, а не баг продукта. Лекция 3 это уже доказала; CamoLeak — её dev-инстанс. Защита та же, не новая.

Каждый риск — и его конкретный контроль.

Точка	Риск	Контроль (часто не-AI)
1 · §1.4	«почти правильный» код	человеческое ревью + тест до доверия фрагменту
2 · §2.3	merge без чтения, рост техдолга	ревью PR + CI-gate (клоны / churn / mutation)
3 · §3.4–3.5	деструктив без гейта; perception-gap	hard human-gate на необратимое; измерять, не ощущать
4 · §4.4/4.7	уязвимый код + ложная уверенность; утечка	SAST/secret-scan; least-privilege + изоляция + egress

Каждый риск имеет конкретный, известный, часто не-AI контроль. Задача инженера — не бояться AI и не доверять ему, а знать, какой контроль ставится и почему он не делегируется.

Не все методологии одинаково совместимы с AI.

Заливка: золото SOLID = лучшая совместимость; бирюза SOLID = антипаттерн, отвергается по построению.

Методология	Почему ложится / что меняется
TDD — №1	тест пишется до кода = точная исполняемая спецификация + детерминированный feedback-loop для агента (не самооценка)
spec-driven	контракт (requirements / design / tasks) сужает пространство правдоподобно-неверного
trunk-based + CI-gates	компенсация системного эффекта: AI ↑throughput, но ↓delivery stability (DORA)
vibe-coding — антипаттерн	без структуры/ограничений/теста/гейта — снимает все 3 опоры паттерна §1.5; корень провалов лекции

vibe-coding (строгое определение): генерировать и принимать код «по ощущению» — без явной структуры задачи, без сформулированных ограничений, без машинно-проверяемого теста и без quality-gate, доверяя правдоподобию вывода. Это не методология, а её отсутствие.

DORA 2025: «AI — амплификатор, и TDD усиливается». Индустрия отвергает не AI, а паттерн vibe-coding-без-гейтов.

Практики уточняются, не уходят.

Brooks, «No Silver Bullet» (1986)

accidental (привнесённая): рутина, boilerplate, ручной debug, доки — AI силен

essential (существенная): «the hardest part is deciding precisely what to build» — AI не помогает

→ AI удешевил принесённую; существенная — постановка, выбор, ответственность — осталась там же

DORA 2025 (n~5000)

- adoption ~90%, широкая вера в рост продуктивности
- отрицательная связь AI с delivery stability — 2-й год

«AI doesn't fix a team; it amplifies what's already there» — слабый процесс деградирует быстрее

AI — амплификатор, не корректор. Сильная культура с AI сильнее; слабая — деградирует быстрее. Критерий: рабочие gates ДО масштабирования AI, а не вместо.

Конфигурация — это размен под задачу, не «что круче».

solo + AI

дёшево / быстро, нет coordination overhead

«exhausted bottleneck»: одно узкое место 24/7, нет второго ревьюера → single point of failure

команда + AI

peer-review, распределённая ответственность, ownership

AI амплифицирует сильную команду (и слабую — DORA)

solo + AI

ранняя стадия / MVP / well-scoped / обратимые последствия

команда + AI

прод / регулируемое / долгоживущий код / нужен аудит и распределённая ответственность

Ландшафт (направление, не точные доли): Copilot стагнирует (всё ещё #1 по охвату); Claude Code / Cursor растут; уходит не инструмент, а практика vibe-coding-без-гейтов.

Разделяйте подтверждённое и заявленное.

docs-as-code — документация в репозитории, под версионным контролем, рядом с кодом и в том же ревью.

ПОДТВЕРЖДЕНО (HIGH)

- AGENTS.md / CLAUDE.md — машиночитаемый контекст агента — де-факто стандарт
- формализован 08.2025, рост ~20k → 40k+ репозиторий, нативная поддержка
- работает как контекст-инжиниринг для агента (мост Лекции 3)

СЛАБО ПОДТВЕРЖДЕНО

- «спека замещает код как единственный источник истины» = vendor-claim
- сами SDD-практики: «code remains the source of truth»
- числа 3–10× — early-adopter-репорты, не независимое исследование

Первое — использовать. Второе — не закладывать в архитектуру как факт. Та же дисциплина, что с benchmark-числами: отделять подтверждённое от заявленного.

AGENTS.md/CLAUDE.md adoption-числа [VFY-day-of]. «Спека = единственная истина» — vendor-claim, пометать «слабо подтверждено».

Матрица выбора уровня автономии.

Структура для аргумента, не сумма баллов. Бирюза SOLID = жёсткий гейт / слабая сторона.

Ось ↓	 А автодоп	 В мелкие	 С агент	 D оркестр.
Незнакомость кода	любая	средняя ок	высокая → строгий ревью	высокая → не D
Обратимость	любая	обратимое	необратимое → ревью	только обратимое / hard-gate
Критичность / прод	любая (с чтением)	некрит.-средн.	критичное → senior	прод → hard human-gate
Аудит / ответств.	человек на токене	человек на фрагменте	человек на merge	человек на approval+merge+прод
Цена ошибки	строгость чтения	необходимость теста	строгость ревью/CI	допустим ли D вообще



Детерминированная, верифицируемая, повторяемая задача (парсинг, валидация по схеме, арифметика, маршрутизация по правилам) → обычный код, без AI. AI добавил бы недетерминизм + «почти правильное» + поверхность prompt injection без выигрыша.

Когда правильный ответ — понизить автономию или не AI.



1. Детерминированная верифицируемая → не AI вовсе

парсинг, валидация по схеме, арифметика, маршрутизация —
обычный код точен и аудируем (§5.2)



2. High-stakes без ревью → недопустимо

необратимое/критичное/прод без человеческого гейта — профиль
Replit/Kiro/PocketOS



3. Обучение junior делегированием → вредит навыку

Anthropic RCT: делегировавшие — -17% на квизе; спрашивавшие
«как работает» — без деградации



4. Автономия без hard-гейта на необратимое → запрещено

уровень D без least-privilege / rollback / egress — не настраиваемый
параметр

Глава не «бойтесь AI» и не «AI всё решит». Между AI-карго-культом и AI-отрицанием — инженер, который для каждой задачи называет уровень, конфигурацию, точку человеческого контроля и условие смены.

Чек-лист «прежде чем дать задачу AI».

- 1 Можно ли решить без AI (детерм., верифиц.)? → не добавляй AI
- 2 Обратимо ли последствие? Необратимое → hard human-gate
- 3 Есть ли тест-оракул? Нет → ревью обязательно
- 4 Кто ревьюит и кто мержит? Merge — всегда человек
- 5 Затронуты секреты / недоверенный контент? → least-priv + изоляция
- 6 Насколько код знаком AI? Незнакомый → доверие по Pro (~64%)
- 7 Цель — артефакт или навык? Навык → не делегировать генерацию
- 8 Конфигурация: solo+AI или команда+AI — по обратимости/аудиту?

Worked example (задача A)

приватный платёжный модуль, тесты неполные → С с обязательным senior-ревью + hard-гейт, команда+AI. Решающая ось — необратимость+критичность. Условие смены: обратимый внутренний инструмент с полным покрытием → допустим D.

Mini-apply (задача B)

миграция формата конфигов в ~300 репо по фикс-правилу, мержит человек — пройдите чек-лист сами за 2 минуты (think-pair-share)

Сначала попытка — потом сверка. Сформулируйте ответ ДО разбора. Полная отработка — Семинар 4.



Лекция 4 — первая отраслевая

Лестница A → D + матрица §6.1 + чек-лист §6.4 — линза для всех следующих отраслевых лекций: тот же вопрос в новой обёртке.

Задание — Семинар 4: «AI в цикле разработки ПО: автодополнение, чат-ассистент, агент»

Для каждого кейса: уровень + ≥ 2 причины по осям + ≥ 1 условие смены + явное «где человек обязателен». Mini-apply задача B — разминка.

Вопросы

Спасибо за внимание

Семинар 4 — на следующей неделе. Дополнительные вопросы — на e-mail.